

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and [Amazon](http://Amazon.com).

Regression

```
# install empiricaldist if necessary
```

```
try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py
```

```
from os.path import basename, exists
```

```
def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)
```

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

In the previous chapter we saw several examples of logistic regression, which is based on the assumption that the likelihood of an outcome, expressed in the form of log odds, is a linear function of some quantity (continuous or discrete).

In this chapter we'll work on examples of simple linear regression, which models the relationship between two quantities. Specifically, we'll look at changes over time in snowfall and the marathon world record.

The models we'll use have three parameters, so you might want to review the tools we used for the three-parameter model in <<_MarkandRecapture>>.

More Snow?

I am under the impression that we don't get as much snow around here as we used to. By "around here" I mean Norfolk County, Massachusetts, where I was born, grew up, and currently live. And by "used to" I mean compared to when I was young, like in 1978 when we got 27 inches of snow and I didn't have to go to school for a couple of weeks.

Fortunately, we can test my conjecture with data. Norfolk County happens to be the location of the Blue Hill Meteorological Observatory, which keeps the oldest continuous weather record in North America.

Data from this and many other weather stations is available from the National Oceanic and Atmospheric Administration (NOAA). I collected data from the Blue Hill Observatory from May 11, 1967 to May 11, 2020.

To get more data, go to National Oceanic and Atmospheric Administration, select daily summaries, choose a date range, and search for Stations with search term "Blue Hill Coop". Add it to the cart.

Open cart and select "Custom GHCN-Daily CSV", then continue.

Select all data types (but particularly Precipitation) and continue.

Provide an email address and submit order.

You'll get an email with a download link. Download the CSV file and move it into the current directory.

The following cell downloads the data as a CSV file.

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/data/2239075.csv')
```

Downloaded 2239075.csv

We can use Pandas to read the data into `DataFrame` :

```
import pandas as pd

df = pd.read_csv('2239075.csv', parse_dates=[2])
```

Here's what the last few rows look like.

```
df.tail(3)
```

	STATION	NAME	DATE	PRCP	SNOW	SNWD	TM
19357	USC00190736	BLUE HILL COOP, MA US	2020- 05-09	0.45	0.0	0.0	
19358	USC00190736	BLUE HILL COOP, MA US	2020- 05-10	0.00	0.0	0.0	
19359	USC00190736	BLUE HILL COOP, MA US	2020- 05-11	0.00	0.0	0.0	

The columns we'll use are:

- `DATE`, which is the date of each observation,
- `SNOW`, which is the total snowfall in inches.

I'll add a column that contains just the year part of the dates.

```
df['YEAR'] = df['DATE'].dt.year
```

And use `groupby` to add up the total snowfall in each year.

```
snow = df.groupby('YEAR')['SNOW'].sum()
```

The first and last years are not complete, so I'll drop them.

```
snow = snow.iloc[1:-1]
len(snow)
```

52

The following figure shows total snowfall during each of the complete years in my lifetime.

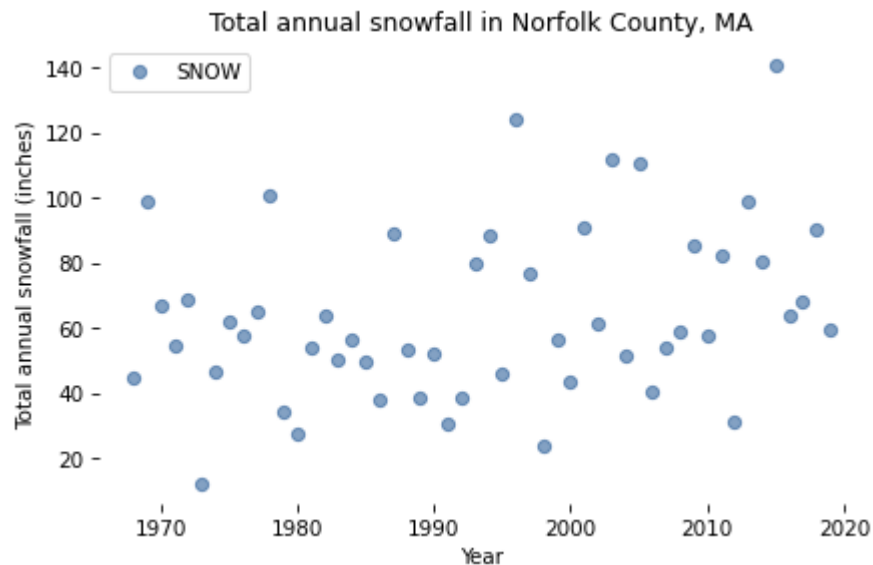
```

from utils import decorate

snow.plot(ls='', marker='o', alpha=0.5)

decorate(xlabel='Year',
        ylabel='Total annual snowfall (inches)',
        title='Total annual snowfall in Norfolk County, MA')

```



Looking at this plot, it's hard to say whether snowfall is increasing, decreasing, or unchanged. In the last decade, we've had several years with more snow than 1978, including 2015, which was the snowiest winter in the Boston area in modern history, with a total of 141 inches.

This kind of question -- looking at noisy data and wondering whether it is going up or down -- is precisely the question we can answer with Bayesian regression.

```
snow.loc[[1978, 1996, 2015]]
```

```

YEAR
1978    100.6
1996    124.2
2015    141.1
Name: SNOW, dtype: float64

```

Regression Model

The foundation of regression (Bayesian or not) is the assumption that a time series like this is the sum of two parts:

1. A linear function of time, and
2. A series of random values drawn from a distribution that is not changing over time.

Mathematically, the regression model is

$$y = ax + b + \epsilon$$

where y is the series of measurements (snowfall in this example), x is the series of times (years) and ϵ is the series of random values.

a and b are the slope and intercept of the line through the data. They are unknown parameters, so we will use the data to estimate them.

We don't know the distribution of ϵ , so we'll make the additional assumption that it is a normal distribution with mean 0 and unknown standard deviation, σ .

To see whether this assumption is reasonable, I'll plot the distribution of total snowfall and a normal model with the same mean and standard deviation.

Here's a `Pmf` object that represents the distribution of snowfall.

```
from empiricaldist import Pmf

pmf_snowfall = Pmf.from_seq(snow)
```

And here are the mean and standard deviation of the data.

```
mean, std = pmf_snowfall.mean(), pmf_snowfall.std()
mean, std
```

```
(np.float64(64.19038461538462), np.float64(26.28802198439569))
```

I'll use the `norm` object from SciPy to compute the CDF of a normal distribution with the same mean and standard deviation.

```
from scipy.stats import norm

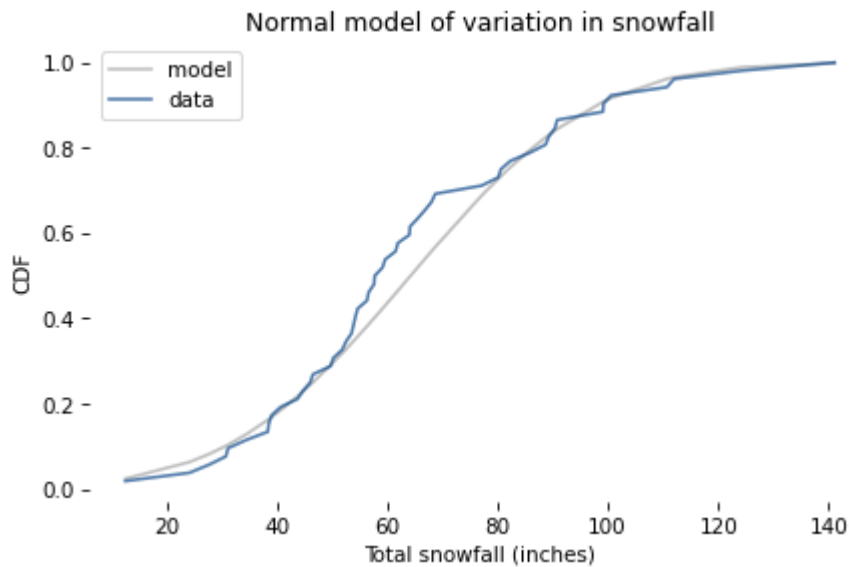
dist = norm(mean, std)
qs = pmf_snowfall.qs
ps = dist.cdf(qs)
```

Here's what the distribution of the data looks like compared to the normal model.

```
import matplotlib.pyplot as plt

plt.plot(qs, ps, color='C5', label='model')
pmf_snowfall.make_cdf().plot(label='data')

decorate(xlabel='Total snowfall (inches)',
         ylabel='CDF',
         title='Normal model of variation in snowfall')
```



We've had more winters below the mean than expected, but overall this looks like a reasonable model.

Least Squares Regression

Our regression model has three parameters: slope, intercept, and standard deviation of ϵ . Before we can estimate them, we have to choose priors. To help with that, I'll use StatsModel to fit a line to the data by least squares regression.

First, I'll use `reset_index` to convert `snow`, which is a `Series`, to a `DataFrame`.

```
data = snow.reset_index()
data.head(3)
```

	YEAR	SNOW
0	1968	44.7
1	1969	99.2
2	1970	66.8

The result is a `DataFrame` with two columns, `YEAR` and `SNOW`, in a format we can use with `StatsModels`.

As we did in the previous chapter, I'll center the data by subtracting off the mean.

```
offset = round(data['YEAR'].mean())
data['x'] = data['YEAR'] - offset
offset
```

```
1994
```

And I'll add a column to `data` so the dependent variable has a standard name.

```
data['y'] = data['SNOW']
```

Now, we can use `StatsModels` to compute the least squares fit to the data and estimate `slope` and `intercept`.

```
import statsmodels.formula.api as smf

formula = 'y ~ x'
results = smf.ols(formula, data=data).fit()
results.params
```

```
Intercept    64.446325
x             0.511880
dtype: float64
```

The intercept, about 64 inches, is the expected snowfall when `x=0`, which is the beginning of 1994. The estimated slope indicates that total snowfall is increasing at a rate of about 0.5 inches per year.

`results` also provides `resid`, which is an array of residuals, that is, the differences between the data and the fitted line. The standard deviation of the residuals is an estimate of `sigma`.

```
results.resid.std()
```

```
np.float64(25.385680731210623)
```

We'll use these estimates to choose prior distributions for the parameters.

Priors

I'll use uniform distributions for all three parameters.

```
import numpy as np
from utils import make_uniform

qs = np.linspace(-0.5, 1.5, 51)
prior_slope = make_uniform(qs, 'Slope')
```

```
qs = np.linspace(54, 75, 41)
prior_inter = make_uniform(qs, 'Intercept')
```

```
qs = np.linspace(20, 35, 31)
prior_sigma = make_uniform(qs, 'Sigma')
```

I made the prior distributions different lengths for two reasons. First, if we make a mistake and use the wrong distribution, it will be easier to catch the error if they are all different lengths.

Second, it provides more precision for the most important parameter, `slope`, and spends less computational effort on the least important, `sigma`.

In `<<_ThreeParameterModel>>` we made a joint distribution with three parameters. I'll wrap that process in a function:

```
from utils import make_joint

def make_joint3(pmf1, pmf2, pmf3):
    """Make a joint distribution with three parameters."""
    joint2 = make_joint(pmf2, pmf1).stack()
    joint3 = make_joint(pmf3, joint2).stack()
    return Pmf(joint3)
```

And use it to make a `Pmf` that represents the joint distribution of the three parameters.

```
prior = make_joint3(prior_slope, prior_inter, prior_sigma)
prior.head(3)
```

			probs
Slope	Intercept	Sigma	
-0.5	54.0	20.0	0.000015
		20.5	0.000015
		21.0	0.000015

The index of `Pmf` has three columns, containing values of `slope`, `inter`, and `sigma`, in that order.

With three parameters, the size of the joint distribution starts to get big. Specifically, it is the product of the lengths of the prior distributions. In this example, the prior distributions have 51, 41, and 31 values, so the length of the joint prior is 64,821.

```
len(prior_slope), len(prior_inter), len(prior_sigma)
```

```
(51, 41, 31)
```

```
len(prior_slope) * len(prior_inter) * len(prior_sigma)
```

```
64821
```

```
len(prior)
```

```
64821
```

Likelihood

Now we'll compute the likelihood of the data. To demonstrate the process, let's assume temporarily that the parameters are known.

```
inter = 64  
slope = 0.51  
sigma = 25
```

I'll extract the `xs` and `ys` from `data` as `Series` objects:

```
xs = data['x']  
ys = data['y']
```

And compute the "residuals", which are the differences between the actual values, `ys`, and the values we expect based on `slope` and `inter`.

```
expected = slope * xs + inter  
resid = ys - expected
```

According to the model, the residuals should follow a normal distribution with mean 0 and standard deviation `sigma`. So we can compute the likelihood of each residual value using `norm` from SciPy.

```
densities = norm(0, sigma).pdf(resid)
```

The result is an array of probability densities, one for each element of the dataset; their product is the likelihood of the data.

```
likelihood = densities.prod()  
likelihood
```

```
np.float64(1.3551948769060997e-105)
```

As we saw in the previous chapter, the likelihood of any particular dataset tends to be small. If it's too small, we might exceed the limits of floating-point arithmetic. When that happens, we can avoid the problem by computing likelihoods under a log transform. But in this example that's not necessary.

The Update

Now we're ready to do the update. First, we need to compute the likelihood of the data for each possible set of parameters.

```
likelihood = prior.copy()  
  
for slope, inter, sigma in prior.index:  
    expected = slope * xs + inter  
    resid = ys - expected  
    densities = norm.pdf(resid, 0, sigma)  
    likelihood[slope, inter, sigma] = densities.prod()
```

This computation takes longer than many of the previous examples. We are approaching the limit of what we can do with grid approximations.

Nevertheless, we can do the update in the usual way:

```
posterior = prior * likelihood  
posterior.normalize()
```

```
np.float64(6.769832641515687e-107)
```

The result is a `Pmf` with a three-level index containing values of `slope`, `inter`, and `sigma`. To get the marginal distributions from the joint posterior, we can use `Pmf.marginal`, which we saw in `<<_ThreeParameterModel>>`.

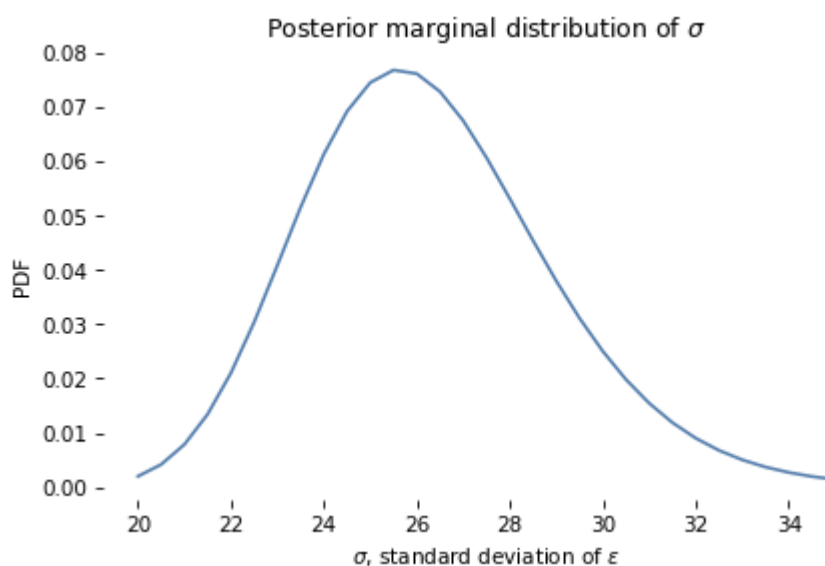
```
posterior_slope = posterior.marginal(0)
posterior_inter = posterior.marginal(1)
posterior_sigma = posterior.marginal(2)
```

Here's the posterior distribution for `sigma`:

```
posterior_sigma.plot()

decorate(xlabel='$\sigma$, standard deviation of $\epsilon$',
        ylabel='PDF',
        title='Posterior marginal distribution of $\sigma$')
```

```
<>:3: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the fu
ture. Did you mean "\\s"? A raw string is also an option.
<>:5: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the fu
ture. Did you mean "\\s"? A raw string is also an option.
<>:3: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the fu
ture. Did you mean "\\s"? A raw string is also an option.
<>:5: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the fu
ture. Did you mean "\\s"? A raw string is also an option.
/var/folders/jy/hjxktrkx6734j6m4b4rt51ww0000gn/T/ipykernel_97258/3121049885.py:3: SyntaxWarnin
g: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean
"\s"? A raw string is also an option.
    decorate(xlabel='$\sigma$, standard deviation of $\epsilon$',
/var/folders/jy/hjxktrkx6734j6m4b4rt51ww0000gn/T/ipykernel_97258/3121049885.py:5: SyntaxWarnin
g: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean
"\s"? A raw string is also an option.
    title='Posterior marginal distribution of $\sigma$')
```



The most likely values for `sigma` are near 26 inches, which is consistent with our estimate based on the standard deviation of the data.

However, to say whether snowfall is increasing or decreasing, we don't really care about `sigma`. It is a "nuisance parameter", so-called because we have to estimate it as part of the model, but we don't need it to answer the questions we are interested in.

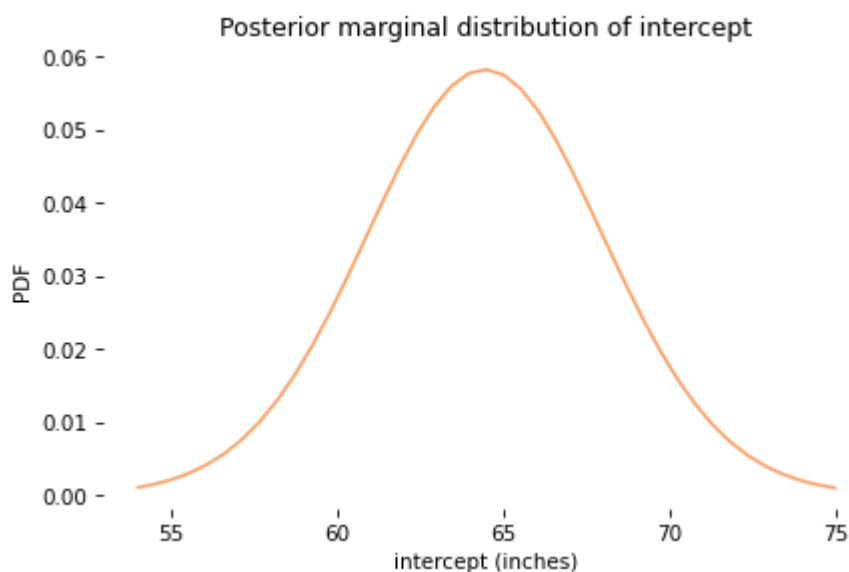
Nevertheless, it is good to check the marginal distributions to make sure

- The location is consistent with our expectations, and
- The posterior probabilities are near 0 at the extremes of the range, which indicates that the prior distribution covers all parameters with non-negligible probability.

In this example, the posterior distribution of `sigma` looks fine.

Here's the posterior distribution of `inter` :

```
posterior_inter.plot(color='C1')
decorate(xlabel='intercept (inches)',
        ylabel='PDF',
        title='Posterior marginal distribution of intercept')
```



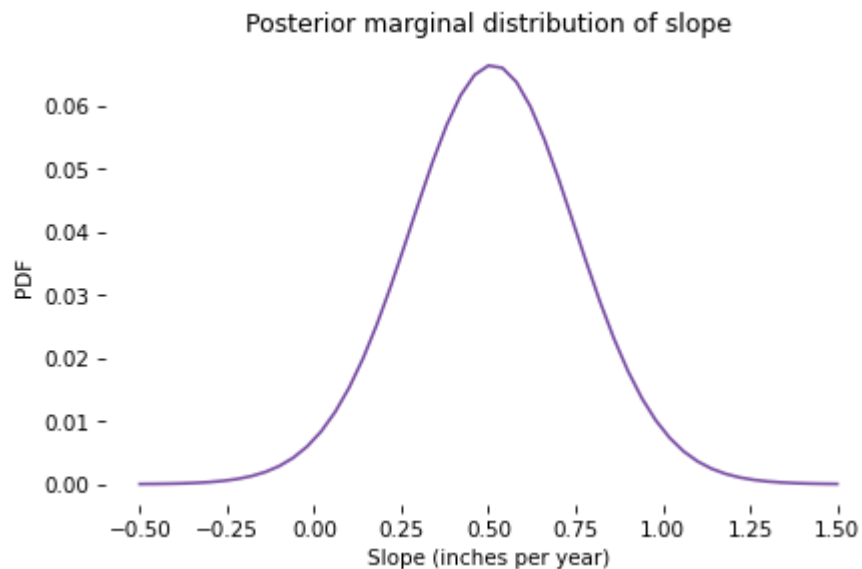
```
from utils import summarize
summarize(posterior_inter)
```

```
64.448 [58.725 70.275]
```

The posterior mean is about 64 inches, which is the expected amount of snow during the year at the midpoint of the range, 1994.

And finally, here's the posterior distribution of `slope` :

```
posterior_slope.plot(color='C4')
decorate(xlabel='Slope (inches per year)',
         ylabel='PDF',
         title='Posterior marginal distribution of slope')
```



```
summarize(posterior_slope)
```

```
0.512 [0.1 0.9]
```

The posterior mean is about 0.51 inches, which is consistent with the estimate we got from least squared regression.

The 90% credible interval is from 0.1 to 0.9, which indicates that our uncertainty about this estimate is pretty high. In fact, there is still a small posterior probability (about 2%) that the slope is negative.

```
posterior_slope.make_cdf()(0)
```

```
array(0.01840519)
```

However, it is more likely that my conjecture was wrong: we are actually getting more snow around here than we used to, increasing at a rate of about a half-inch per year, which is substantial. On average, we get an additional 25 inches of snow per year than we did when I was young.

This example shows that with slow-moving trends and noisy data, your instincts can be misleading.

Now, you might suspect that I overestimate the amount of snow when I was young because I enjoyed it, and underestimate it now because I don't. But you would be mistaken.

During the Blizzard of 1978, we did not have a snowblower and my brother and I had to shovel. My sister got a pass for no good reason. Our driveway was about 60 feet long and three cars wide near the garage. And we had to shovel Mr. Crocker's driveway, too, for which we were not allowed to accept payment. Furthermore, as I recall it was during this excavation that I accidentally hit my brother with a shovel on the head, and it bled a lot because, you know, scalp wounds.

Anyway, the point is that I don't think I overestimate the amount of snow when I was young because I have fond memories of it.

Optimization

The way we computed the likelihood in the previous section was pretty slow. The problem is that we looped through every possible set of parameters in the prior distribution, and there were more than 60,000 of them.

If we can do more work per iteration, and run the loop fewer times, we expect it to go faster.

In order to do that, I'll unstack the prior distribution:

```
joint3 = prior.unstack()
joint3.head(3)
```

Slope	Sigma	20.0	20.5	21.0	21.5	22.0
	Intercept					
-0.5	54.000	0.000015	0.000015	0.000015	0.000015	0.000015
	54.525	0.000015	0.000015	0.000015	0.000015	0.000015
	55.050	0.000015	0.000015	0.000015	0.000015	0.000015

3 rows × 31 columns

The result is a `DataFrame` with `slope` and `intercept` down the rows and `sigmas` across the columns.

The following is a version of `likelihood_regression` that takes the joint prior distribution in this form and returns the posterior distribution in the same form.

```

from utils import normalize

def update_optimized(prior, data):
    """Posterior distribution of regression parameters
    `slope`, `inter`, and `sigma`.

    prior: Pmf representing the joint prior
    data: DataFrame with columns `x` and `y`

    returns: Pmf representing the joint posterior
    """
    xs = data['x']
    ys = data['y']
    sigmas = prior.columns
    likelihood = prior.copy()

    for slope, inter in prior.index:
        expected = slope * xs + inter
        resid = ys - expected
        resid_mesh, sigma_mesh = np.meshgrid(resid, sigmas)
        densities = norm.pdf(resid_mesh, 0, sigma_mesh)
        likelihood.loc[slope, inter] = densities.prod(axis=1)

    posterior = prior * likelihood
    normalize(posterior)
    return posterior

```

This version loops through all possible pairs of `slope` and `inter`, so the loop runs about 2000 times.

```
len(prior_slope) * len(prior_inter)
```

```
2091
```

Each time through the loop, it uses a grid mesh to compute the likelihood of the data for all values of `sigma`. The result is an array with one column for each data point and one row for each value of `sigma`. Taking the product across the columns (`axis=1`) yields the probability of the data for each value of `sigma`, which we assign as a row in `likelihood`.

```
%time posterior_opt = update_optimized(joint3, data)
```

```
CPU times: user 336 ms, sys: 1.28 ms, total: 338 ms
Wall time: 338 ms
```

We get the same result either way.

```
np.allclose(posterior, posterior_opt.stack())
```

True

But this version is about 25 times faster than the previous version.

This optimization works because many functions in NumPy and SciPy are written in C, so they run fast compared to Python. If you can do more work each time you call these functions, and less time running the loop in Python, your code will often run substantially faster.

In this version of the posterior distribution, `slope` and `inter` run down the rows and `sigma` runs across the columns. So we can use `marginal` to get the posterior joint distribution of `slope` and `intercept`.

```
from utils import marginal

posterior2 = marginal(posterior_opt, 1)
posterior2.head(3)
```

		probs
Slope	Intercept	
-0.5	54.000	7.633362e-08
	54.525	1.013295e-07
	55.050	1.327249e-07

The result is a `Pmf` with two columns in the index. To plot it, we have to unstack it.

```
joint_posterior = posterior2.unstack().transpose()
joint_posterior.head(3)
```

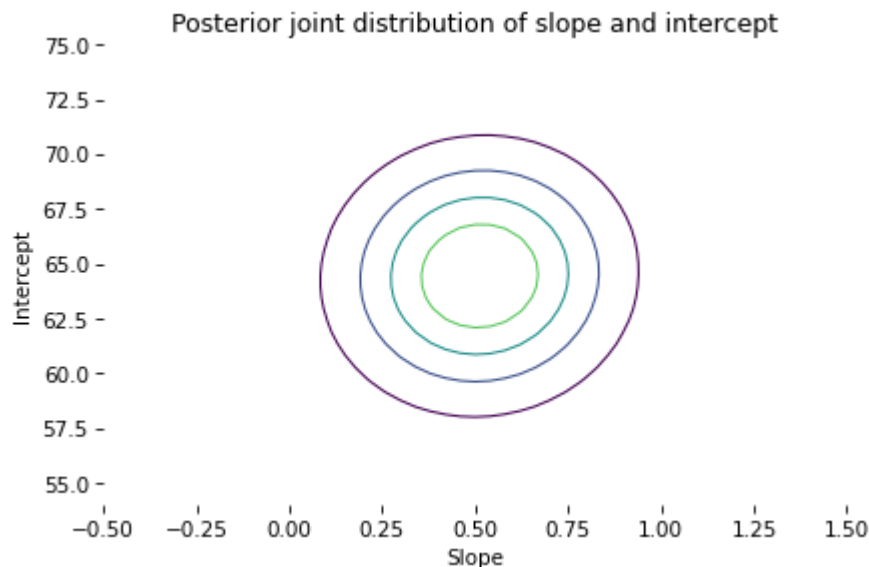
Slope	-0.50	-0.46	-0.42	-0.38	-0.34
Intercept					
54.000	7.633362e-08	1.244120e-07	1.999617e-07	3.168007e-07	4.945131e-07
54.525	1.013295e-07	1.658787e-07	2.678095e-07	4.262364e-07	6.684267e-07
55.050	1.327249e-07	2.182169e-07	3.538722e-07	5.657543e-07	8.912797e-07

3 rows × 51 columns

Here's what it looks like.

```
from utils import plot_contour

plot_contour(joint_posterior)
decorate(title='Posterior joint distribution of slope and intercept')
```



The ovals in the contour plot are aligned with the axes, which indicates that there is no correlation between `slope` and `inter` in the posterior distribution, which is what we expect since we centered the values.

In this example, the motivating question is about the slope of the line, so we answered it by looking at the posterior distribution of slope.

In the next example, the motivating question is about prediction, so we'll use the joint posterior distribution to generate predictive distributions.

Marathon World Record

For many running events, if you plot the world record pace over time, the result is a remarkably straight line. People, including me, have speculated about possible reasons for this phenomenon.

People have also speculated about when, if ever, the world record time for the marathon will be less than two hours. (Note: In 2019 Eliud Kipchoge ran the marathon distance in under two hours, which is an astonishing achievement that I fully appreciate, but for several reasons it did not count as a world record).

So, as a second example of Bayesian regression, we'll consider the world record progression for the marathon (for male runners), estimate the parameters of a linear model, and use the model to predict when a runner will break the two-hour barrier.

The following cell downloads a web page from Wikipedia that includes a table of marathon world records, and uses Pandas to put the data in a `DataFrame`.

```
url = "https://github.com/AllenDowney/ThinkBayes2/raw/master/data/Marathon_world_record_prog
```

```
tables = pd.read_html(url)
len(tables)
```

```
5
```

The first table is the one we want.

```
table = tables[0]
table.tail(3)
```

	Time	Name	Nationality	Date	Event/Place	Source
47	2:03:23	Wilson Kipsang	Kenya	September 29, 2013	Berlin Marathon	IAAF[83][84] ARRS[82]
48	2:02:57	Dennis Kimetto	Kenya	September 28, 2014	Berlin Marathon	IAAF[85][86] ARRS[82]
49	2:01:39	Eliud Kipchoge	Kenya	September 16, 2018	Berlin Marathon	IAAF[1

We can use Pandas to parse the dates. A few of them include notes that cause parsing problems, but the argument `errors='coerce'` tells Pandas to fill invalid dates with `NaT`, which is a version of `NaN` that represents "not a time".

```
table['date'] = pd.to_datetime(table['Date'], errors='coerce')
table['date'].head()
```

```
0    1908-07-24
1    1909-01-01
2    1909-02-12
3    1909-05-08
4             NaT
Name: date, dtype: datetime64[us]
```

We can also use Pandas to parse the record times.

```
table['time'] = pd.to_timedelta(table['Time'])
```

And convert the times to paces in miles per hour.

```
table['y'] = 26.2 / table['time'].dt.total_seconds() * 3600
table['y'].head()
```

```
0    8.967143
1    9.099504
2    9.419942
3    9.465508
4    9.672854
Name: y, dtype: float64
```

The following function plots the results.

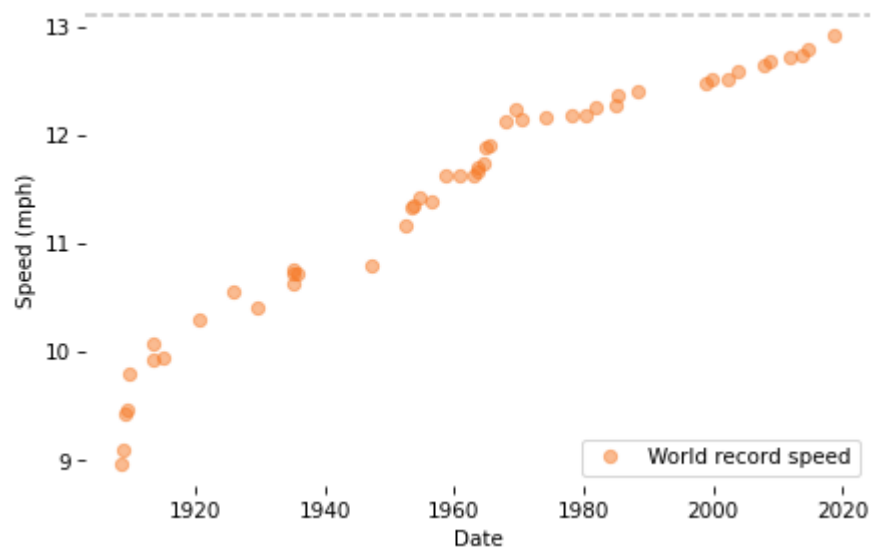
```
def plot_speeds(df):
    """Plot marathon world record speed as a function of time.

    df: DataFrame with date and mph
    """
    plt.axhline(13.1, color='C5', ls='--')
    plt.plot(df['date'], df['y'], 'o',
             label='World record speed',
             color='C1', alpha=0.5)

    decorate(xlabel='Date',
            ylabel='Speed (mph)')
```

Here's what the results look like. The dashed line shows the speed required for a two-hour marathon, 13.1 miles per hour.

```
plot_speeds(table)
```



It's not a perfectly straight line. In the early years of the marathon, the record speed increased quickly; since about 1970, it has been increasing more slowly.

For our analysis, let's focus on the recent progression, starting in 1970.

```
recent = table['date'] > pd.to_datetime('1970')
data = table.loc[recent].copy()
data.head()
```

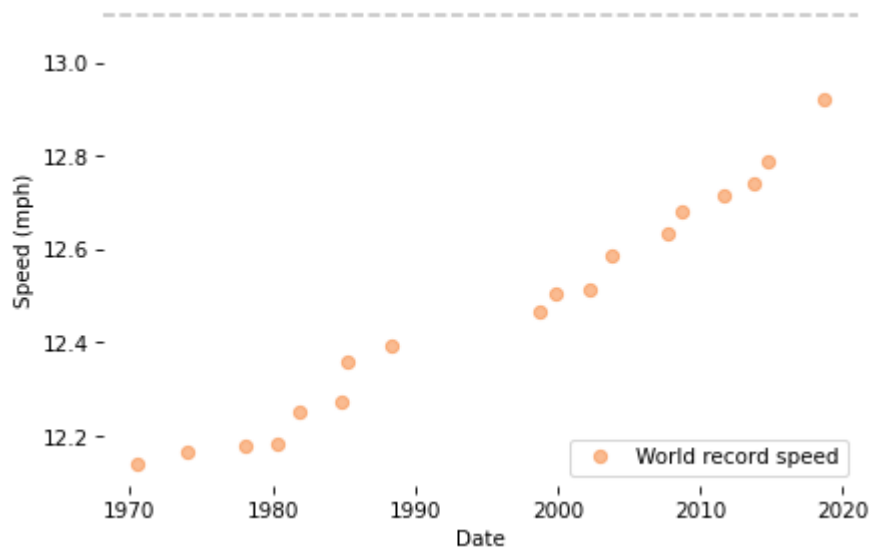
	Time	Name	Nationality	Date	Event/Place	Source
32	2:09:28.8	Ron Hill	United Kingdom	July 23, 1970	Edinburgh, Scotland	ARRS[
33	2:09:12	Ian Thompson	United Kingdom	January 31, 1974	Christchurch, New Zealand	ARRS[
34	2:09:05.6	Shigeru So	Japan	February 5, 1978	Beppu-Ōita Marathon	ARRS[
35	2:09:01	Gerard Nijboer	Netherlands	April 26, 1980	Amsterdam Marathon	ARRS[
36	2:08:18	Robert De Castella	Australia	December 6, 1981	Fukuoka Marathon	IAA [5] ARRS[

In the notebook for this chapter, you can see how I loaded and cleaned the data. The result is a `DataFrame` that contains the following columns (and additional information we won't use):

- `date`, which is a Pandas `Timestamp` representing the date when the world record was broken, and
- `speed`, which records the record-breaking pace in mph.

Here's what the results look like, starting in 1970:

```
plot_speeds(data)
```



The data points fall approximately on a line, although it's possible that the slope is increasing.

To prepare the data for regression, I'll subtract away the approximate midpoint of the time interval, 1995.

```
offset = pd.to_datetime('1995')
timedelta = table['date'] - offset
```

When we subtract two `Timestamp` objects, the result is a "time delta", which we can convert to seconds and then to years.

```
data['x'] = timedelta.dt.total_seconds() / 3600 / 24 / 365.24
```

```
data['x'].describe()
```

```
count    18.000000
mean      0.740913
std       15.417918
min      -24.444201
25%      -12.352152
50%       4.264319
75%      13.492498
max       23.707699
Name: x, dtype: float64
```

As in the previous example, I'll use least squares regression to compute point estimates for the parameters, which will help with choosing priors.

```
import statsmodels.formula.api as smf

formula = 'y ~ x'
results = smf.ols(formula, data=data).fit()
results.params
```

```
Intercept    12.460507
x             0.015464
dtype: float64
```

The estimated intercept is about 12.5 mph, which is the interpolated world record pace for 1995. The estimated slope is about 0.015 mph per year, which is the rate the world record pace is increasing, according to the model.

Again, we can use the standard deviation of the residuals as a point estimate for `sigma`.

```
results.resid.std()
```

```
np.float64(0.0413996122019321)
```

These parameters give us a good idea where we should put the prior distributions.

The Priors

Here are the prior distributions I chose for `slope`, `intercept`, and `sigma`.

```
qs = np.linspace(0.012, 0.018, 51)
prior_slope = make_uniform(qs, 'Slope')
```

```
qs = np.linspace(12.4, 12.5, 41)
prior_inter = make_uniform(qs, 'Intercept')
```

```
qs = np.linspace(0.01, 0.21, 31)
prior_sigma = make_uniform(qs, 'Sigma')
```

And here's the joint prior distribution.

```
prior = make_joint3(prior_slope, prior_inter, prior_sigma)
prior.head()
```

			probs
Slope	Intercept	Sigma	
0.012	12.4	0.010000	0.000015
		0.016667	0.000015
		0.023333	0.000015

Now we can compute likelihoods as in the previous example:

```
xs = data['x']
ys = data['y']
likelihood = prior.copy()

for slope, inter, sigma in prior.index:
    expected = slope * xs + inter
    resid = ys - expected
    densities = norm.pdf(resid, 0, sigma)
    likelihood[slope, inter, sigma] = densities.prod()
```

Now we can do the update in the usual way.

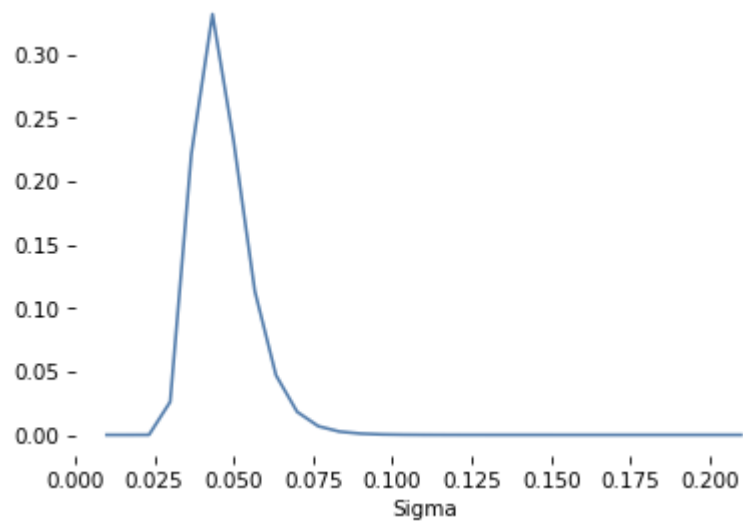
```
posterior = prior * likelihood
posterior.normalize()
```

```
np.float64(654100803618.6724)
```

And unpack the marginals:

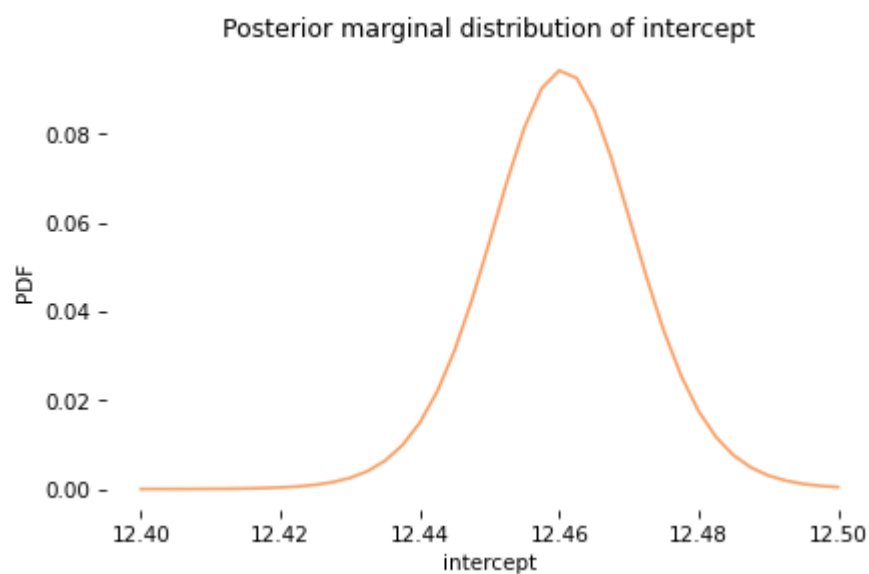
```
posterior_slope = posterior.marginal(0)
posterior_inter = posterior.marginal(1)
posterior_sigma = posterior.marginal(2)
```

```
posterior_sigma.plot();
```



Here's the posterior distribution of `inter` :

```
posterior_inter.plot(color='C1')  
decorate(xlabel='intercept',  
         ylabel='PDF',  
         title='Posterior marginal distribution of intercept')
```



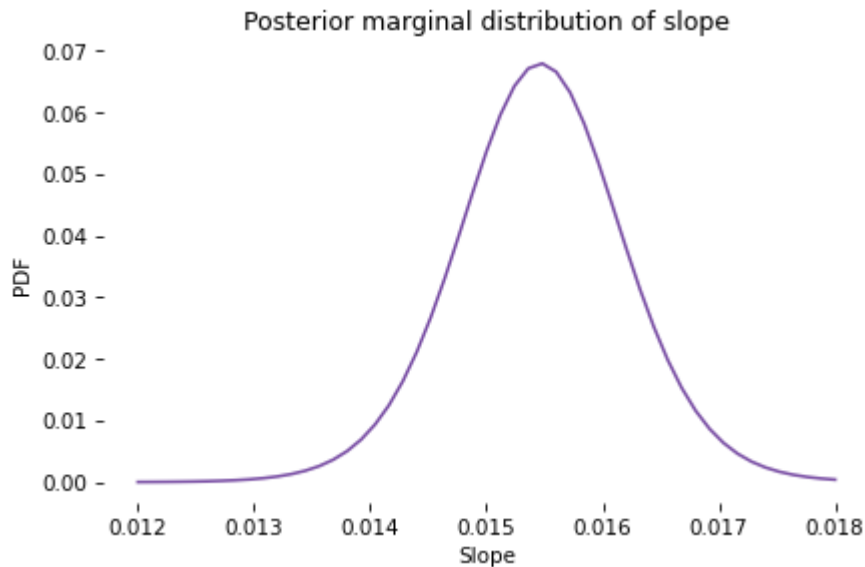
```
summarize(posterior_inter)
```

```
12.46 [12.4425 12.4775]
```

The posterior mean is about 12.5 mph, which is the world record marathon pace the model predicts for the midpoint of the date range, 1994.

And here's the posterior distribution of `slope`.

```
posterior_slope.plot(color='C4')
decorate(xlabel='Slope',
         ylabel='PDF',
         title='Posterior marginal distribution of slope')
```



```
summarize(posterior_slope)
```

```
0.015 [0.01428 0.01668]
```

The posterior mean is about 0.015 mph per year, or 0.15 mph per decade.

That's interesting, but it doesn't answer the question we're interested in: When will there be a two-hour marathon? To answer that, we have to make predictions.

Prediction

To generate predictions, I'll draw a sample from the posterior distribution of parameters, then use the regression equation to combine the parameters with the data.

`Pmf` provides `choice`, which we can use to draw a random sample with replacement, using the posterior probabilities as weights.

```
np.random.seed(17)
```

```
sample = posterior.choice(101)
```

The result is an array of tuples. Looping through the sample, we can use the regression equation to generate predictions for a range of `xs`.

```
xs = np.arange(-25, 50, 2)
pred = np.empty((len(sample), len(xs)))

for i, (slope, inter, sigma) in enumerate(sample):
    epsilon = norm(0, sigma).rvs(len(xs))
    pred[i] = inter + slope * xs + epsilon
```

Each prediction is an array with the same length as `xs`, which I store as a row in `pred`. So the result has one row for each sample and one column for each value of `x`.

We can use `percentile` to compute the 5th, 50th, and 95th percentiles in each column.

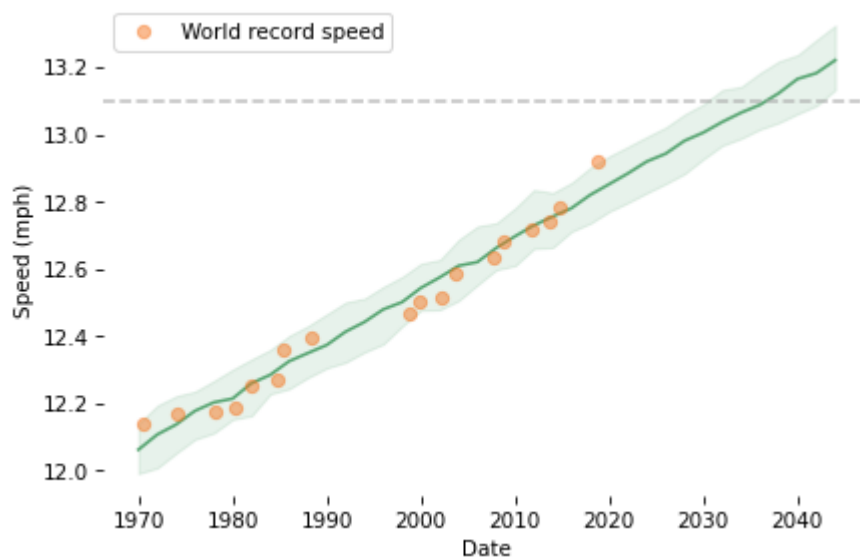
```
low, median, high = np.percentile(pred, [5, 50, 95], axis=0)
```

To show the results, I'll plot the median of the predictions as a line and the 90% credible interval as a shaded area.

```
times = pd.to_timedelta(xs*365.24, unit='days') + offset

plt.fill_between(times, low, high,
                 color='C2', alpha=0.1)
plt.plot(times, median, color='C2')

plot_speeds(data)
```



The dashed line shows the two-hour marathon pace, which is 13.1 miles per hour. Visually we can estimate that the prediction line hits the target pace between 2030 and 2040.

To make this more precise, we can use interpolation to see when the predictions cross the finish line. SciPy provides `interp1d`, which does linear interpolation by default.

```
from scipy.interpolate import interp1d

future = np.array([interp1d(high, xs)(13.1),
                   interp1d(median, xs)(13.1),
                   interp1d(low, xs)(13.1)])

dts = pd.to_timedelta(future*365.24, unit='day') + offset
pd.DataFrame(dict(datetime=dts),
              index=['early', 'median', 'late'])
```

	datetime
early	2030-06-21 12:40:01.592340389
median	2036-09-20 09:03:28.522400448
late	2042-08-13 11:20:06.313348412

The median prediction is 2036, with a 90% credible interval from 2032 to 2043. So there is about a 5% chance we'll see a two-hour marathon before 2032.

Summary

This chapter introduces Bayesian regression, which is based on the same model as least squares regression; the difference is that it produces a posterior distribution for the parameters rather than point estimates.

In the first example, we looked at changes in snowfall in Norfolk County, Massachusetts, and concluded that we get more snowfall now than when I was young, contrary to my expectation.

In the second example, we looked at the progression of world record pace for the men's marathon, computed the joint posterior distribution of the regression parameters, and used it to generate predictions for the next 20 years.

These examples have three parameters, so it takes a little longer to compute the likelihood of the data. With more than three parameters, it becomes impractical to use grid algorithms.

In the next few chapters, we'll explore other algorithms that reduce the amount of computation we need to do a Bayesian update, which makes it possible to use models with more parameters.

But first, you might want to work on these exercises.

Exercises

Exercise: I am under the impression that it is warmer around here than it used to be. In this exercise, you can put my conjecture to the test.

We'll use the same dataset we used to model snowfall; it also includes daily low and high temperatures in Norfolk County, Massachusetts during my lifetime.

Here's the data.

```
df = pd.read_csv('2239075.csv', parse_dates=[2])
df.head(3)
```

	STATION	NAME	DATE	PRCP	SNOW	SNWD	TMAX
0	USC00190736	BLUE HILL COOP, MA US	1967- 05-11	0.43	0.0	0.0	57
1	USC00190736	BLUE HILL COOP, MA US	1967- 05-12	0.00	0.0	0.0	58
2	USC00190736	BLUE HILL COOP, MA US	1967- 05-13	0.00	0.0	0.0	64

Again, I'll create a column that contains the year part of the dates.

```
df['YEAR'] = df['DATE'].dt.year
```

This dataset includes `TMIN` and `TMAX`, which are the daily low and high temperatures in degrees F. I'll create a new column with the daily midpoint of the low and high temperatures.

```
df['TMID'] = (df['TMIN'] + df['TMAX']) / 2
```

Now we can group by year and compute the mean of these daily temperatures.

```
tmid = df.groupby('YEAR')['TMID'].mean()  
len(tmid)
```

54

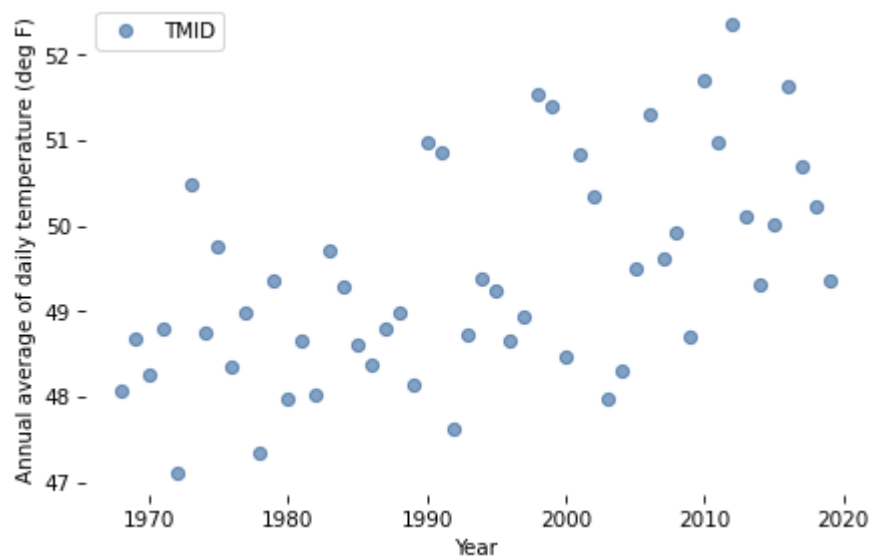
Again, I'll drop the first and last years, which are incomplete.

```
complete = tmid.iloc[1:-1]  
len(complete)
```

52

Here's what the time series looks like.

```
complete.plot(ls='', marker='o', alpha=0.5)  
  
decorate(xlabel='Year',  
         ylabel='Annual average of daily temperature (deg F)')
```



As we did with the snow data, I'll convert the `Series` to a `DataFrame` to prepare it for regression.

```
data = complete.reset_index()  
data.head()
```

	YEAR	TMID
0	1968	48.071038
1	1969	48.687671
2	1970	48.258904
3	1971	48.804110
4	1972	47.112022

```
offset = round(data['YEAR'].mean())
offset
```

```
1994
```

```
data['x'] = data['YEAR'] - offset
data['x'].mean()
```

```
np.float64(-0.5)
```

```
data['y'] = data['TMID']
data['y'].std()
```

```
np.float64(1.2389114009625752)
```

Now we can use StatsModels to estimate the parameters.

```
import statsmodels.formula.api as smf

formula = 'y ~ x'
results = smf.ols(formula, data=data).fit()
results.params
```

```
Intercept    49.430172
x             0.044252
dtype: float64
```

And compute the standard deviation of the parameters.

```
results.resid.std()
```

```
np.float64(1.0417057653902053)
```

According to the least squares regression model, annual average temperature is increasing by about 0.044 degrees F per year.

To quantify the uncertainty of these parameters and generate predictions for the future, we can use Bayesian regression.

1. Use StatsModels to generate point estimates for the regression parameters.
2. Choose priors for `slope`, `intercept`, and `sigma` based on these estimates, and use `make_joint3` to make a joint prior distribution.
3. Compute the likelihood of the data and compute the posterior distribution of the parameters.
4. Extract the posterior distribution of `slope`. How confident are we that temperature is increasing?
5. Draw a sample of parameters from the posterior distribution and use it to generate predictions up to 2067.
6. Plot the median of the predictions and a 90% credible interval along with the observed data.

Does the model fit the data well? How much do we expect annual average temperatures to increase over my (expected) lifetime?

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)